# LakeTemperature Testing Software

Slides by Aidan Bensch and Sam Allen

# Background

- We are testing a module of *E3SM,*[1] an environmental model. This module, `LakeTemperature`, is part of the greater *ELM* (*E3SM* Land Model) which was originally a fork of the *CLM* (Community Land Model).
- We are able to test specifically this module using *SPEL,*[2] a software that adds a *NetCDF*[3] interface for the constants, inputs, and outputs of a module of *E3SM* and compiles that specific part of the model. The executable produced by this is named `elmtest.exe`.

1. "Energy Exascale Earth System Model." *E3SM*, U.S. Department of Energy: Office of Science, n.d., e3sm.org/.

2. Schwartz, Peter. "SPEL_OpenACC." *GitHub*, 16 Oct. 2025, github.com/peterdschwartz/SPEL_OpenACC.git.

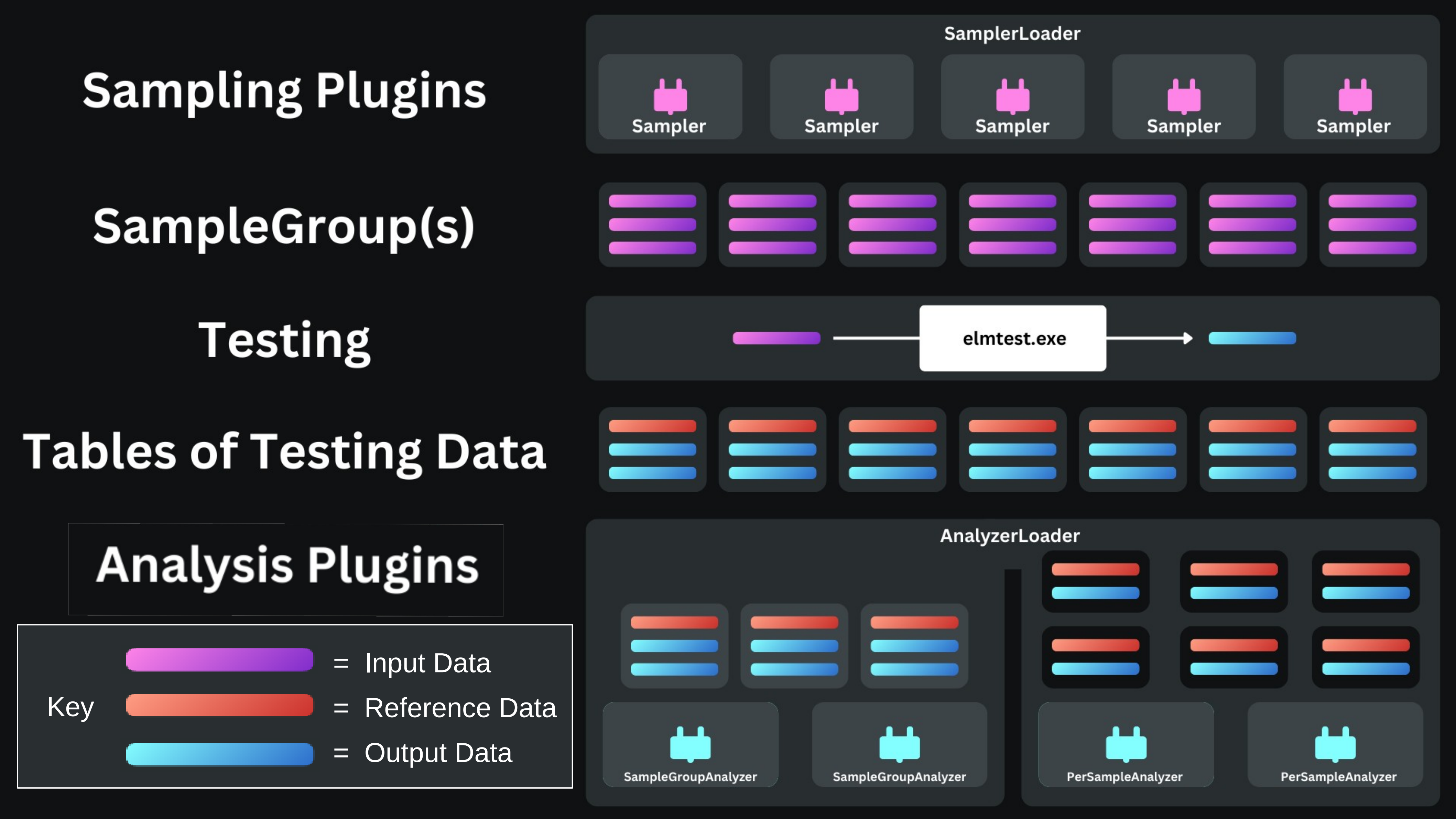
3. "NetCDF." *NSF UNIDATA*, n.d., www.unidata.ucar.edu/software/netcdf.

# Project Goals

- Develop a software that can systematically change the constants used in the `LakeTemperature` model and allow us to visualize and observe patterns in how this affects its output.
- Apply testing techniques to try and find values for constants that will cause physically impossible or otherwise erroneous outputs. Apply assertions pertaining to the values of constants, inputs, and outputs based around physical laws and other constraints provided by domain scientists.

# Our Testing Solution

- Automates the process of modifying constants in `spel-constants0001.nc` and the process of analyzing the resulting values in `fut-outputs0001.nc`
- Both the process of choosing values for each constant and the process of analyzing the resulting outputs is extendable through sampling plugins and analysis plugins respectively.



Sampling Plugins
SampleGroup(s)
Testing
Tables of Testing Data
Analysis Plugins
Key
= Input Data
= Reference Data
= Output Data
SamplerLoader
Sampler
Sampler
Sampler
Sampler
Sampler
elmtest.exe
AnalyzerLoader
SampleGroupAnalyzer
SampleGroupAnalyzer
PerSampleAnalyzer
PerSampleAnalyzer

# Testing Expanded

## Sample Generation

Using a Sampling Plugin to create samples
with unique values for the 25 scalar constants.

## Running elmtest.exe

Running the program using the constants
and recording the results.

## Analysis

Using an Analysis Plugin to analyze the results,
especially in comparison to reference data.

# Constants, Inputs, and Outputs

- The 28 constants control how the model behaves and manipulates data.
  - We don’t yet support modification of the 3 array constants. Of the other 25 scalar constants we do experiment with, 8 have no observable effect on the output.
- The 26 inputs provide starting data for the model.
- The 9 outputs include data that the model has generated.
- 8 variables are both an input and an output in some instances.

# Table of Constants, Inputs, and Outputs

| | |
|---|---|
| Array Constants | crop |
| | iscft |
| | percrop |
| Scalar Constants (with observable effects) | betavis |
| | cnfac |
| | cpliq |
| | denh2o |
| | dtime_mod |
| | grav |
| | hfus |
| | lake_no_ed |
| | lakepuddling |

# Table of Constants, Inputs, and Outputs

| | |
|---|---|
| Scalar Constants (with observable effects) | nlevgrnd |
| | nlevlak |
| | nlevsno |
| | nlevsoi |
| | thk_bedrock |
| | tkwat |
| | use_lch4 |
| | vkc |
| Scalar Constants (without observable effects) | cpice |
| | denice |
| | depthcrit |
| | iulog |

# Table of Constants, Inputs, and Outputs

| | |
|---|---|
| Scalar Constants (without observable effects) | mixfact |
| | pudz |
| | tkair |
| | tkice |
| Inputs | betaprime_col |
| | eflx_gnet |
| | eflx_sh_grnd |
| | eflx_sh_tot |
| | eflx_snomelt |
| | eflx_soil_grnd |
| | errsoi |
| | grnd_ch4_cond_col |

# Table of Constants, Inputs, and Outputs

| | |
|---|---|
| Inputs | hc_soi |
| | hc_soisno |
| | imelt |
| | lakeresist_col |
| | lake_icefrac_col |
| | lake_icethick_col |
| | qflx_snofrz_lyr |
| | qflx_snomelt |
| | qflx_snow_melt |
| | savedtke1_col |
| | h2osno |
| | snow_depth |

# Table of Constants, Inputs, and Outputs

| | |
|---|---|
| Inputs | t_lake |
| | t_soisno |
| | |
| | |
| | |
| | |
| Inputs & Outputs | |
| | |
| | |
| | |
| | |
| | |

# Table of Constants, Inputs, and Outputs

| Inputs & Outputs | |
|---|---|
| | |
| Outputs | ch4_vars |
| | col_ef |
| | col_es |
| | col_wf |
| | col_ws |
| | lakestate_vars |
| | veg_ef |
| | |
| | |

# *NetCDF*

- Network Common Data Form (*NetCDF*) is a format for self-describing datasets. Each file is composed of “dimensions” and “variables.” Variables are multi-dimensional arrays, with the meaning of each dimension described in the file.

```
netcdf x_times_y {
dimensions:
        x = 5 ;
        y = 5 ;
variables:
        float x(x) ;
        float y(y) ;
        float z(x, y) ;
data:

 x = -2, -1, 0, 1, 2 ;

 y = -4, -2, 0, 2, 4 ;

 z =
  8, 4, -0, -4, -8,
  4, 2, -0, -2, -4,
  -0, -0, 0, 0, 0,
  -4, -2, 0, 2, 4,
  -8, -4, 0, 4, 8 ;
}
```

# What is a Sample?

- A sample is a set of values for the constants contained in `spel-constants0001.nc`.
  - ~~There are 28 constants in total. We don't yet support modification of the 3 array constants. Of the 25 scalar constants we experiment with, 8 have no observable effect on the output.~~
- `elmtest.exe` is run once for each sample.

# What is a Sample? (Illustration)

Constants (formerly lakeparams.txt)

```
betavis
0.0
za_lake
0.6
n2min
7.4999999999999993E-005
tdmax
277.0
pudz
0.0
mixfact
10.0
depthcrit
25.0
```

Example Sample (JSON used for illustration)



```json
{
  "betavis": 0.0,
  "za_lake": 0.6,
  "n2min": 7.4999999999999993E-005,
  "tdmax": 277.0,
  "pudz": 0.0,
  "mixfact": 10.0,
  "depthcrit": 25.0
}
```

# What Outputs are Received per Sample?

- Analysis plugins will receive the outputs from the `spel-outputs0001.nc` file in the form of multidimensional numpy arrays, which range from one to three dimensions.

Outputs Received by Analysis Plugins (JSON for illustration)

```
{
  "veg_ef%eflx_soil_grnd": [0.0, ...],
  "veg_ef%eflx_gnet": [[0.0, ...], ...],
  ...
}
```

# Sample Groups

- Represents a grouping of samples, ordered—for the sake of simplicity, not necessity—to match indices with the list of model data
- Analysis plugins can analyze the result of multiple consecutive runs of `elmtest.exe` in context this way.
- Having multiple sample groups is useful because each one can cover a range of values for a specific combination of constants whose combined effect is being examined.

# Sample Group Illustration

Sample Group Data (JSON for illustration)

```
[
  {
    "betavis": 0.0,
    ...
  },
  {
    "betavis": 0.25,
    ...
  },
  {
    "betavis": 0.5,
    ...
  }
]
```

Ordered Output Data (JSON for illustration)

```
[
  {
    "lakestate_vars%betaprime_col": [0.0, ...],
    ...
  },
  {
    "lakestate_vars%betaprime_col": [0.25, ...],
    ...
  },
  {
    "lakestate_vars%betaprime_col": [0.5, ...],
    ...
  }
]
```

# Plugins

- Our testing framework supports two types of plugins, sampling and analysis. Sampling plugins define values for constants, and analysis plugins view the outputs of the model.
  - For example, if we have constants A and B, default input data, and a specific output variable O, one question we might have is which of the constants has a greater effect on the output variable O.
  - We could answer this by defining a sampling plugin that would set the constants to be different values, such as A=5 and B=5 for one run, A=10 and B=5 for another run, and A=5 and B=10 for the third run.
  - Then we would need an analysis plugin that focuses on the how the value of output O changes between runs to determine whether constant A or B affects it more.

# Plugin UML Diagram

# Sampling Plugin Interface

- Sampling plugins define a subclass of `Sampler`, returning one or more `SampleGroups`, which are sequences of individual samples.

```
class Sampler(ABC):
    @abstractmethod
    def get_sample_groups(self) -> Mapping[str, SampleGroup]:
        pass
```

# Current Sampling Plugins

- CSV Index Sampling
  - Uses CSV files generated by *ACTS*[1] to create samples. It will take the largest index and use that to linearly interpolate between the values for each constant found in the ranges file
- Order Sampling
  - Uses JSON files called “orders” to create “lines” in the input space (the constants), through linear interpolation

1. “Automated Combinatorial Testing for Software (ACTS).” *National Institute of Standards and Technology*, 20 July 2016, csrc.nist.rip/groups/SNS/acts/.

# Analysis Plugin Interface

- Analysis plugins can either be “per sample” or “sample group” analysis plugins.
- “Per sample” plugins define a subclass of `PerSampleAnalyzer`, whose methods are called each time `elmtest.exe` is run, and which receives the outputs produced by that specific sample.
- “Sample group” plugins define a subclass of `SampleGroupAnalyzer`, whose method is called after all samples in a group are run through `elmtest.exe`, and which receives all outputs from each.

# Analysis Plugin Interface Visuals

```python
class PerSampleAnalyzer(ABC):
    @abstractmethod
    def analyze_sample_data(
        self,
        sample: Sample,
        reference_data: ModelData,
        test_data: ModelData,
    ) -> tuple[str, FileSystemTree]:
        pass
```

```python
class SampleGroupAnalyzer(ABC):
    @abstractmethod
    def analyze_sample_data(
        self,
        sample_group: SampleGroup,
        reference_data: ModelData,
        sample_data: list[ModelData],
    ) -> tuple[str, FileSystemTree]:
        pass
```

# Current Analysis Plugins

- Change Detection
  - Identifies, per sample group, which inputs and outputs differ at some point from the reference
- Differences
  - Finds differences between values in the reference data and the current sample's data. It records the indices of the difference, the two differing values, and the difference between them.
- Fault Finding
  - Uses "check" functions defined by the plugin user to make assertions about values in the sample data, similar to a unit testing framework but with assertions pertaining to model values rather than specific function behavior
- NetCDF4 Output
  - Stores each sample group as an individual NetCDF file, to be used for graphing changes based on our samples. To do this, it just adds a new dimension "sample_index" and layers each sample's NetCDF file on top of the others using that.

# Configuration